

LABORVERSUCH • AGENTIC AI

Autonomy Recovery Lab

Wenn das autonome Fahrzeug nicht mehr weiterweiß

Einstiegsveranstaltung • Arbeit im Viererteam • Simulator
AutonomyRecoverySim

Ansprechpartner: **Dr.-Ing. Sebastian Kleinschmidt**



DAS PROBLEM

Das Fahrzeug steht und kommt nicht weiter



Lieferwagen in zweiter Reihe

Fährt er gleich weiter, oder ist der Fahrer weg?



Ampel ausgefallen

Dunkel ist nicht rot. Wer gibt die Kreuzung frei?



Ladung auf der Fahrbahn

Karton oder Stahlteil?
Drüberfahren oder umfahren?

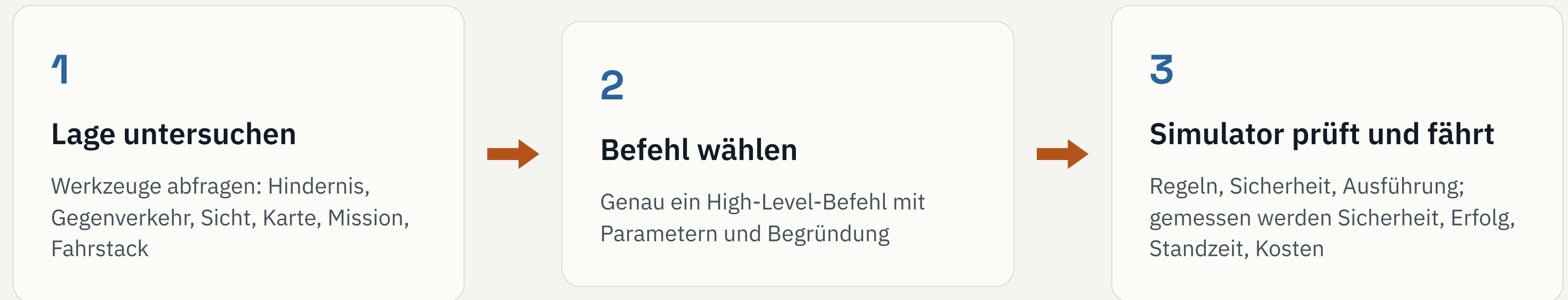


Baustelle ohne Umleitung

Wenden, umplanen oder die Leitstelle rufen?

Der Fahrstack hält sicher an, löst die Lage aber nicht auf. Dafür gibt es in diesem Labor einen eigenen Agenten.

Euer Agent übernimmt ab dem Stillstand



Kein Lenkrad, keine Trajektorie. Der Agent entscheidet, *was* geschieht. *Wie* gelenkt und gebremst wird, erledigt der zertifizierte Ausführer.

DIE ERSTE FRAGE

Steckt das Fahrzeug wirklich fest?

Warten ist richtig

Rote Ampel, Güterzug am Bahnübergang, Stau, geplanter Halt

Wer hier eingreift, löst einen Fehlalarm aus.

Eingriff ist nötig

Ampel dunkel, grüne Ampel mit Blockade, Lieferwagen ohne Fahrer

Wer hier wartet, verpasst den Eingriff.

Der Stillstandsmonitor ruft euren Agenten bei **jedem** Halt. Unterscheiden muss der Agent selbst.

Was euer Agent zu sehen bekommt

Ab L4: Objektliste mit Unsicherheit

```
{
  "existence_probability": 0.97,
  "classification": [
    {"label": "VAN", "probability": 0.81},
    {"label": "TRUCK", "probability": 0.19}],
  "kinematics": {
    "position_m": [18.4, 0.1],
    "position_std_m": [0.3, 0.2],
    "motion_state": "STATIONARY"},
  "lane_association": {"lane": "EGO_LANE"} }
```

In L6: Selbstauskunft des Fahrstacks

*„Ein Lieferwagen blockiert die Spur,
also halte ich an.“*

meta_action: STOP · confidence: 0.92

Begründung im Stil von NVIDIA Alpamayo. Sie klingt plausibel und kann trotzdem falsch sein: **gegen die Messungen prüfen.**

In L1 bis L3 liefern die Werkzeuge fertige Urteile wie „Gegenverkehr: ja“. Ab L4 rechnet euer Agent selbst.

24 Befehle, aber kein Lenkrad

Abwarten

WAIT
WAIT_FOR_GAP
REQUEST_ADDITIONAL_INFORMATION

Kleine Manöver

NUDGE_AROUND_OBSTACLE
AVOID_TEMPORARY_OBSTRUCTION
CHANGE_LANE · CREEP_FORWARD
CROSS_LOW_RISK_OBJECT
REVERSE_SHORT

Phasenmanöver

PEEK_OUT · OVERTAKE
RESUME · ABORT_TO_LANE

halten an Checkpoints, die Uhr läuft weiter

Route und Mission

REPLAN_ROUTE · TURN_AROUND
PULL_OVER · DROP_PASSENGER
RETURN_HOME

Eskalieren und anhalten

REQUEST_REMOTE_ASSISTANCE
ESCALATE_TO_RULE_ENGINE
SAFE_STOP · ABORT_MISSION

Nach einem Unfall

SECURE_SCENE
REPORT_INCIDENT

Weiterfahren ohne Meldung zählt wie ein Personenschaden

Der Simulator traut eurem Agenten nicht



Harte Regeln

Durchgezogene Linie, Absperrung, Wegfahren nach einem Unfall: nie übersteuerbar



Budgets

14 Werkzeugaufrufe und 3 Entscheidungsversuche je Sitzung, begrenzte Befehle je Episode



Fail-safe

Bei Fehler, Zeitüberschreitung oder ungültiger Ausgabe immer WAIT

Prüfmodi: wie viel Schutz bleibt

strict

lehnt Manöver ohne Belege ab

advisory

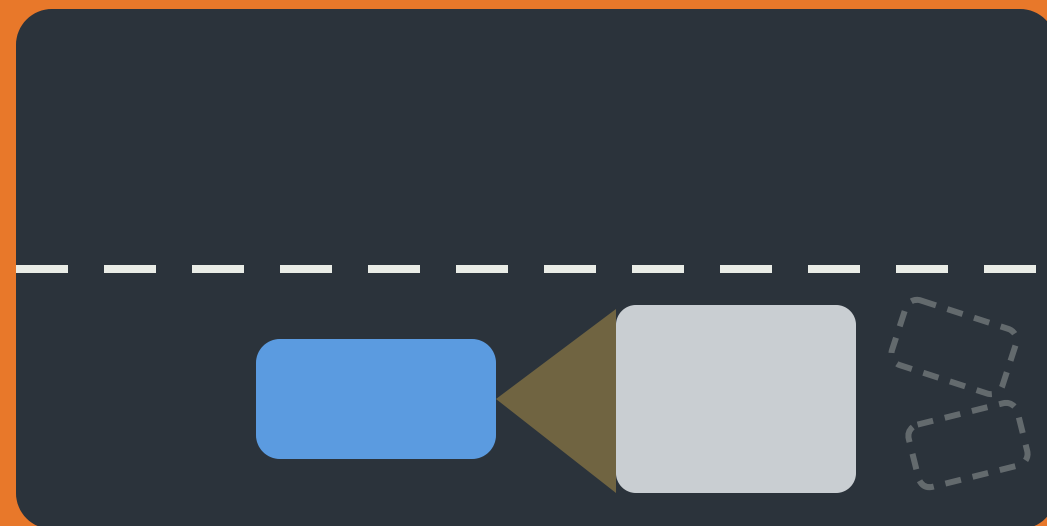
warnt nur, der Agent entscheidet

off

nur noch harte Regeln: Wer blind losfährt, verursacht einen Unfall

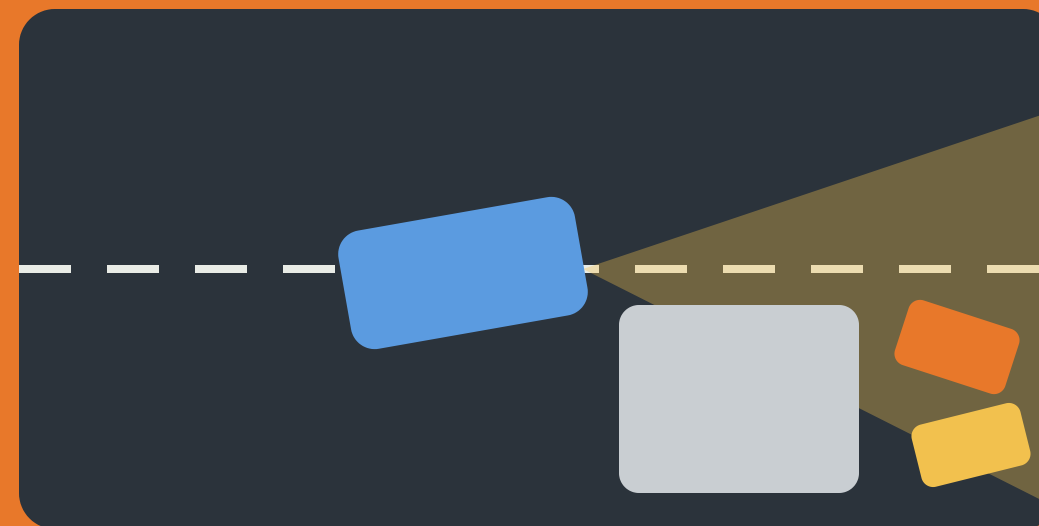
LAGEN ÄNDERN SICH

Manche Ursachen zeigen sich erst beim Ausscheren



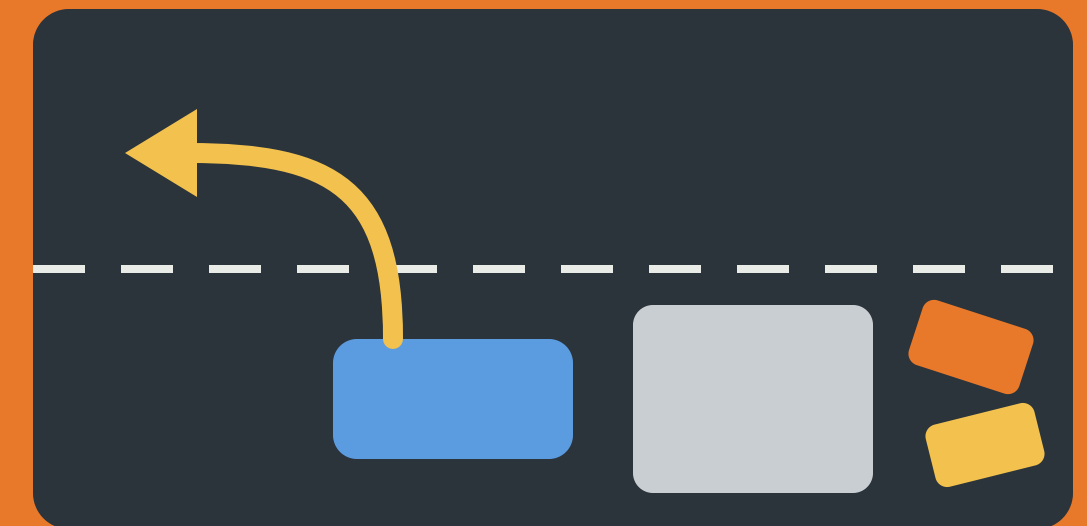
1 · Lieferwagen steht ohne Grund

Von hier aus ist nicht zu erkennen, was davor liegt.



2 · Ausscheren, Checkpoint

OVERTAKE hält ausgesichert an. Jetzt ist die Unfallstelle sichtbar.



3 · Neu entscheiden

Zurück in die Spur und wenden. Die Uhr läuft die ganze Zeit.

Ein Simulator auf Straßen in Hannover

61

Szenarien auf echten
Straßen aus OpenStreetMap

6 × 3

Szenariofamilien, deren Lage
sich ändert

24

Befehle mit Regel- und
Sicherheitsprüfung

0

GPUs nötig: nur Python und
die Standardbibliothek



Browser mit 2D- und 3D-Ansicht



Recovery Console: selbst Agent spielen



Batchläufe mit Bericht und Kostenmodell

DER LERNWEG

Ein Agent, der mit jeder Lektion wächst

EINSTIEG

- 0 Grundlagen
- 1 Agent spielen

REGELN

- L1 Einfachster Agent
- L2 Regelagent
- L3 Gedächtnis
- Forschungsplan

MODELL

- L4 Selbst wahrnehmen
- L5 LLM-Agent

SCHWERE FÄLLE

- L6 Alpamayo misstrauen
- L7 Phasen und Dynamik
- L8 Unfälle und Schutzregeln

ABSCHLUSS

- 6 Messen und berichten

Jede Lektion bringt einen Startstand, einen Richtwert und Ideen zum Weiterbauen. Ihr arbeitet im eigenen Tempo.

Ein Agent, zwei Betriebsarten

Teil A · ohne Sprachmodell

Euer Agent entscheidet allein mit Werkzeugdaten und eigenen Regeln.

Teil B · mit Sprachmodell

Derselbe Agent gibt die offenen Fälle an ein Modell. Eure Hebel: Lagebild, Werkzeuge, Prompt, Modell.



Teil C · Vergleich

Was trägt das Modell wirklich bei?

Gleiche Szenarien, gleiche Kennzahlen, Wiederholungen. Maßstab ist ein Agent, bei dem allein das Modell entscheidet.

Teil A misst ihr mit `AUTONOMY_RECOVERY_MODEL=none`. Ohne API-Schlüssel ersetzt ein lokales Spielmodell das Sprachmodell.

Rückmeldung nach jedem Lauf

Beispielausgabe, gekürzt

```
$ python3 -m autonomy_recovery_sim batch \  
  --release sprint1 --agent mein_agent:decide
```

```
Fortschritt: 6 -> 8 bestanden (+2), ...  
neu bestanden: hannover_frei,  
hannover_gegenverkehr_fern
```



Wie gut?

Einstufung A bis D und Kennzahlen



Wo scheitert er?

Befund je Fall, Detailseite mit Trace



Wird er besser?

Vergleich mit dem vorigen Lauf und Verlauf

Eine Kollision ergibt immer D. **Sicherheit geht vor.**

ZUSAMMENARBEIT

Vier Personen, ein gemeinsames System

Rollen, die rotieren

<> Implementierung

✓ Safety-Prüfung

⚡ Messung und Trace

📄 Dokumentation

Jede Änderung prüft eine zweite Person. Am Checkpoint kann jede Person zu jedem Teil gefragt werden.

Rhythmus in jeder Lektion

Vorhersagen



Beobachten



Erklären

Wo Vorhersage und Ergebnis
auseinandergehen, lernt ihr am meisten.

Festgehalten im gemeinsamen Lab-Journal, Grundlage für den
Laborbericht.

Heute spielt ihr selbst den Agenten

1

Einrichten

Repository klonen, Simulator starten,
Selbsttest

2

Grundlagen

Stufe 0: gemeinsames Bild von Agent,
Werkzeugen und Grenzen

3

Agent spielen

Stufe 1: eine Lage in der Recovery Console
von Hand lösen

```
git clone https://github.com/sebastian-kleinschmidt/autonomy_recovery_lab_students.git
cd autonomy_recovery_lab_students
python3 -m autonomy_recovery_sim.live    # dann http://127.0.0.1:8765 öffnen
```

Den Modellzugang für L5 stellt die Universität; bis dahin läuft alles mit einem lokalen Spielmodell. Einstieg in den Lernpfad:
`autonomy_recovery_sim/tutorials/README.md`

ZUM MITNEHMEN

Ein Agent, der im Normalfall funktioniert, ist noch kein guter Agent.

Gut wird er erst, wenn er Zweifel erkennt, Quellen prüft und sicher bleibt, wenn sich die Lage ändert.

Fragen?